

التقرير الفني الشامل لمهندس الواجهات والأنظمة والـ Git

VisionCraft DIP Studio & AR Mirror

دليل مهندس الواجهات وتجربة المستخدم وهندسة البرمجيات

مرجع هندسي تفصيلي يوثق كافة ملفات وكود الطالب أحمد القاضي، استوديو الواقع المعزز (AR Try-On)، سجل استخدام الذكاء الاصطناعي (AI Log Entry)، خريطة الملفات، تحليل ملفات الـ Markdown، ودليل سير العمل عبر Git & GitHub.

الدور التخصصي: مهندس الواجهات وتجربة المستخدم (Frontend Lead)

طلب الدمج الحالي: Pull Request #16 (Status: Open / Merged)

تاريخ التوثيق: 27 سبتمبر 2026م

اسم الطالب (المطور): أحمد القاضي

اسم الفرع المخصص: feature/frontend-studio

المستودع الرسمي: github.com/hrnafifgit/Studio_Image_SE

فهرس محتويات الدليل الفني الشامل

ص 3	1. توثيق استخدام الذكاء الاصطناعي (AI Usage Entry - أحمد القاضي)
ص 4	2. شرح العمل المنفذ وكيف تم بناء المحرك والواجهة خطوة بخطوة
ص 5	3. خريطة ملفات أحمد القاضي في المشروع وأماكنها وأسمائها ووظيفتها
ص 7	4. ميزات وقدرات الواجهة واستوديو الواقع المعزز (AR Try-On Studio)
ص 8	5. خريطة الأكواد وأرقام الأسطر التفصيلية في App.js و Blade
ص 9	6. الأخطاء المكتشفة في الفرونت إند وكيف تم حلها برمجياً
ص 10	7. الشرح المعماري الشامل لكافة ملفات الـ Markdown (.md) في المشروع
ص 12	8. الدليل الكامل للعمل عبر Git و GitHub وتوثيق طلب الدمج #16 PR
ص 14	9. الشرح المعماري لبقية ملفات المشروع (لاراغل، بايثون، وقاعدة البيانات)
ص 15	10. بنك الأسئلة المتوقعة في مناقشة المشروع وإجاباتها النموذجية

1. توثيق استخدام الذكاء الاصطناعي (AI Usage Entry)

وفقاً لمعايير الشفافية والتفكير النقدي المعتمدة في قسم هندسة البرمجيات بجامعة ذمار، يوثق هذا القسم جلسة العصف الذهني واستخدام أداة الذكاء الاصطناعي الخاصة بالطالب أحمد القاضي:

التاريخ (Date):	2026-09-20	اسم الطالب:	أحمد القاضي
الأداة المستخدمة:	Antigravity AI (Google DeepMind)	المسؤولية:	تأسيس المعمارية ومحرك المعالجة والواجهات
الغرض (Purpose):	تصميم هيكل خادم Python Flask وإعداد مسارات الـ API لمعالجة البكسلات وربطها مع Laravel.		
ملخص الطلب (Prompt):	"اقترح هيكلية مجلدات ومسارات RESTful لميكروسيرفيس بايثون يتصل بـ Laravel على المنفذ 5001"		

تحليل الاقتراحات والتفكير النقدي والقرارات الهندسية:

اقتراحات الذكاء الاصطناعي (AI Suggestions)	القرار الهندسي (Decision)	التعليل والمسوغ العلمي (Engineering Justification)
الاقتراح 1: دمج بايثون داخل لارافل مباشرة باستخدام حزمة <code>php-python</code> .	مرفوض (Rejected)	حزم استدعاء بايثون من داخل PHP غير مستقرة، تسبب بطء شديد (Overhead) عند تمرير المصفوفات الضخمة وتتعارض مع ملحقات OpenCV المكتوبة بلغة ++C.
الاقتراح 2: تشغيل Flask كخدمة مصغرة مستقلة على المنفذ 5001 مع مسارات <code>api/process/</code> و <code>api/health/</code> .	مقبول (Accepted)	يحقق مبدأ فصل المسؤوليات (SoC)، خفيف جداً، يتيح معالجة مصفوفات NumPy بدون كتل زائدة، ويوفر سرعة استجابة فائقة.
الاقتراح 3: استخدام إطار عمل Django بدلاً من Flask.	مرفوض (Rejected)	Django إطار ضخ يتضمن ميزات ثقيلة (ORM، لوحة إدارة، قوالب) لا نحتاجها إطلاقاً؛ لأن وظيفة بايثون هنا حسابية علمية بحتة (Microservice).

المراجعة البشرية والتنفيذ العملي (Human Review):

قام الطالب أحمد القاضي باعتماد الهيكل المصغر، وأنشأ ملف الخادم `python_engine/app/server.py`، وقام بتهيئة CORS للسماح بالاتصال، وبرمج مسار فك تشفير سلاسل Base64 مع دعم EXIF Auto-Orientation لتصحيح صور الهواتف الذكية.

2. شرح العمل المنفذ وكيف تم البناء خطوة بخطوة

تم تنفيذ العمل عبر مراحل هندسية متتالية وفق منهجية هندسة البرمجيات:

المرحلة الأولى: تأسيس نواة خادم بايثون ومعالجة البكسلات (Python Microservice)

1. إنشاء خادم Flask: في `python_engine/app/server.py`، تم تجهيز السيرفر ليعمل على المنفذ 5001 مع دعم Flask-CORS و Socket.IO.
2. معالجة الصور الرقمية: كتابة دالة `decode_image_from_base64` التي تستقبل الصور بصيغة Base64، وتصحح اتجاه الصورة تلقائياً عبر EXIF Transpose، وتحولها إلى مصفوفات NumPy ثنائية وثلاثية الأبعاد صالحة لحوال OpenCV.
3. مسار المعالجة `POST /api/process`: يستقبل العملية والبارامترات، يمررها للمحرك الحسابي، ويعيد الصورة الناتجة مشفرة مع المعادلة الرياضية وكود بايثون وزمن التنفيذ بالمللي ثانية.

المرحلة الثانية: بناء وتصميم واجهة الاستوديو التفاعلية (Photoshop Pro Experience)

1. تصميم القالب الأساسي: في `resources/views/welcome.blade.php`، تم بناء واجهة فوتوشوب داكنة بنظام أشرطة الأدوات العائمة (Left Toolbar, Right Panel, Top Menu).
2. لوحة العرض المشطورة (Split Canvas): ربط لوحتي Canvas متطابقتين للمقارنة الحية بنسبة من 0% إلى 100% مع تحكم كامل بالتقريب (Zoom) والتحرك (Pan).
3. المكبر الرقمي 7x7 ومحلل الهستوجرام: برمجة عرض قيم البكسلات الحقيقية ورسم منحنيات كثافة الألوان حياً.

المرحلة الثالثة: تطوير استوديو الواقع المعزز وتجربة الملحقات (AR Virtual Try-On)

1. مودال الملحقات التفاعلي: بناء واجهة مبنية منظمة بشبكة كروت تعرض 5 ملحقات شفافة مصممة بدقة عالية.
2. محرك التراكب الذكي (Interactive Overlay): برمجة إمكانية سحب الملحق، تكبيره وتصغيره بنسب دقيقة، التحكم بنسبة شفافيته، وتثبيتته كطبقة جديدة.
3. نموذج الوجه القياسي: إدراج زر مباشر لتحميل وجه موديل تجريبي لاختبار الملحقات فوراً.

المرحلة الرابعة: التكامل البرمجي والأتمتة الذكية والـ Git

1. الربط الآمن عبر لارافل: إنشاء الدالة المركزية `sendApiProcess` لتوجيه الطلبات عبر مسار لارافل `api/process/` مع حماية CSRF.
2. أتمتة التشغيل الذكي: تعديل `python_engine/run.py` ليكتشف ويشغل لارافل تلقائياً ويفحص قاعدة البيانات وينقل المستخدم فوراً لصفحة التصميم المنفذ 8000.
3. إدارة الـ Git: رفع التعديلات ومزامنة الفرع وحل التعارضات وفتح طلب الدمج الرسمي `Pull Request #16`.

3. خريطة ملفات أحمد القاضي في المشروع وأماكنها ووظيفتها

يوضح هذا الجدول كافة الملفات التي أنشأها أو طورها الطالب أحمد القاضي داخل مستودع المشروع:

اسم الملف (File Name)	المسار الكامل في المستودع	اللغة / النوع	الدور الوظيفي وما كتبه أحمد داخله
welcome.blade.php	resources/views/welcome.blade.php	Blade / HTML5	واجهة الاستوديو الرئيسية، القوائم العلوية، شريط الأدوات، شاشة المقارنة المقسمة، ومودال استوديو الواقع المعزز AR.
app.js	public/js/app.js	JavaScript (ES6+)	محرك الاستوديو التفاعلي: إدارة الـ DOM، دوال الواقع المعزز، دالة الـ API المركزية <code>sendApiProcess</code> ، والقص والرسم.
photoshop.css	public/css/photoshop.css	CSS3 / Dark Theme	تنسيقات الواجهة بالكامل: المظهر الداكن الفاحم، أزرار الأدوات، كروت الملحقات، وتأثيرات الـ Glassmorphism.
gestures.css	public/css/gestures.css	CSS3	تنسيقات مؤشرات تتبع حركة اليد والإيماءات والكاميرا الحية.
server.py	python_engine/app/server.py	Python / Flask	خادم الخدمات المصغرة، فك تشفير Base64، تصحيح EXIF، توجيه الخوارزميات، والتحويل التلقائي (Redirect) للمنفذ 8000.
run.py	python_engine/run.py	Python Core	محرك التشغيل الشامل: يكتشف ويشغل لارافل تلقائياً في الخلفية ويفتح المتصفح فوراً على صفحة التصميم.
start_studio.bat	start_studio.bat/.	Windows Batch Script	سكريبت تنفيذي بنقرة واحدة لتشغيل كامل بيئة المشروع (بايثون ولارافل والمتصفح) بدون كتابة أوامر.
ملحقات الواقع المعزز	public/accessories/*.png	PNG Alpha Images	ملفات صور شفافة عالية الجودة (نظارات شمسية، نظارات طبية، قبعة فيدورا، وشاح شتوي، طقم بدلة رسمية).
صور الاختبار والموديل	public/samples/*.png	PNG Images	صور عينات معالجة الصور الرقمية ونموذج الوجه القياسي <code>portrait_model.png</code> .

4. خريطة الأكواد وأرقام الأسطر التفصيلية في Blade و App.js

جدول دقيق بأرقام الأسطر لتسهيل الاستشهاد البرمجي أثناء العرض والمناقشة:

في ملف الواجهة : `resources/views/welcome.blade.php`

الأسطر	الكتلة البرمجية	الوصف والمحتوى البرمجي
السطر 6	<code>meta name="csrf-token" content="{{
<csrf_token() }}"></code>	توليد رمز الحماية المشفر وتضمينه في رأس الصفحة لحماية كافة طلبات الـ AJAX.
السطر 26	<code></code>	استدعاء شعار النحل الرسمي عبر مساعد الـ Assets في لارافل.
السطر 37	<code>"()onclick="app.openAccessoriesModal</code>	عنصر القائمة العلوية لفتح استوديو الملحقات والواقع المعزز (اختصار Ctrl+Shift+A).
السطر 258	<code>btnToggleAccessories# زر</code>	زر وصول سريع أرجواني بارز في شريط الأدوات العلوي مع إضاءة Glassmorphic.
الأسطر 270 - 320	منطقة الـ Viewport و الـ Canvas	عناصر الـ Raw Canvas و الـ Processed Canvas ومقبض المقارنة المنزلق <code>splitBar#</code> .
الأسطر 1130 - 1175	نافذة المودال <code>accessoriesModal#</code>	الكود الكامل لتنفيذ استوديو الملحقات والواقع المعزز وشبكة كروت الملحقات الخمسة وزر تحميل الموديل.

في ملف الجافاسكربت : `public/js/app.js`

الأسطر	اسم الدالة البرمجية	الوظيفة والتحكم البرمجي
94 - 250	<code>()initDOM</code>	ربط كافة معرّفات الـ DOM (109 معرّف) بما فيها عناصر الملحقات ونواخذ العرض.
253 - 290	<code>()applyAccessoryOverlay</code>	تحميل صورة الملحق الشفافة ورسمها كطبقة تراكب ذكية مع ضبط الإحداثيات والحجم والشفافية.
1340 - 1370	<code>sendApiProcess(payload)</code>	الدالة المركزية لإرسال الطلبات إلى لارافل <code>api/process/</code> مع CSRF وإشعارات Toast.
1740 - 1775	<code>()applyBlendImages</code>	مزج صورتين بنسب خلط مرنة وتحديث مراجع الـ <code>this.formulaBox</code> DOM دون أخطاء.
1890 - 1925	<code>()applyCollageGrid</code>	توليد تصاميم الكولاج وشبكات دمج الصور وتمرير الهوامش وألوان الإطارات.
2015 - 2045	<code>()applyCrop</code>	اقتصاص الصورة بحسب أبعاد وموقع الإطار التفاعلي وإعادة ضبط الـ Canvas.

5. الأخطاء المكتشفة في الفرونت إند وكيف تم حلها هندسياً

1. خطأ تجاوز البوابة وأخطاء الـ CORS (Hardcoded Port 5001):

المشكلة: كان كود الواجهة في دوال الاستبدال والكولاج والدمج يستدعي مباشرة `http://127.0.0.1:5001/api/process` ، مما تسبب في أخطاء حجب الاتصال (CORS Blocked) عند فتح الموقع عبر خادم لاراغل.

الحل الهندسي: تم توحيد كافة الطلبات عبر دالة ذكية `sendApiProcess` توجه الطلب إلى مسار لاراغل الأصلي `api/process/` وتكرر رمز الحماية `X-CSRF-TOKEN` مع توفير خط رجوع تلقائي وتنبيهات Toast تفاعلية للمستخدم.

2. انهيار الواجهة بسبب تعارض مسميات الـ DOM (TypeErrors):

المشكلة: كان الكود يستدعي `this.codeEl` و `this.formulaEl` ، في حين أن العناصر المعرفة في القالب هي `this.codeSnippet` و `this.formulaBox` ، مما سبب توقف السكريبت عند الخطأ: `Cannot set property 'innerText' of undefined`

الحل الهندسي: تم تصحيح المسميات ومطابقتها 100% مع الـ Blade ، والتحقق آلياً من تكامل كافة الـ 109 معرّف ID و 65 دالة تفاعلية.

3. غياب مودال الواقع المعزز (Missing AR Modal):

المشكلة: القائمة العلوية كانت تستدعي دالة `openAccessoriesModal` بينما هيكل المودال غير موجود في صفحة Blade.

الحل الهندسي: تم بناء وتصميم نافذة المودال كاملة وربطها بملحقات المجلد `/public/accessories` .

4. أتمتة تشغيل النظام وتوجيه المستخدم للتصميم مباشرة:

المشكلة: تشغيل بايثون كان يفتح المنفذ 5001 الذي يعطي خطأ 404 لعدم وجود واجهة فيه، وكان يتطلب تشغيل لاراغل يدوياً بأمر ثانٍ.

الحل الهندسي: تم تحديث `python_engine/run.py` ليكتشف ويشغل لاراغل تلقائياً في الخلفية ويفحص قاعدة البيانات، ويفتح المتصفح مباشرة على صفحة التصميم `http://127.0.0.1:8000` .

6. الشرح المعماري الشامل لكافة ملفات الـ (Markdown (.md) في المشروع

تعتبر ملفات الـ (.md) Markdown ركيزة التوثيق والهندسة الاحترافية لأي مشروع مفتوح المصدر على GitHub. يوضح هذا القسم وظيفة كل ملف منها في مشروع VisionCraft بالتفصيل:

اسم الملف (.md)	الهدف والدور في هندسة البرمجيات	المحتوى المكتوب داخله في المشروع
README.md	الواجهة التعريفية الأولى للمشروع (Project Landing Page).	- نبذة عن المشروع وأهدافه العلمية في معالجة الصور الرقمية. - شارات الجودة (CI Workflow Badges, Tech Stack, License). - جدول أعضاء الفريق والمسؤوليات (هارون، أحمد، أمجد، شعيب). - دليل التثبيت والتشغيل السريع خطوة بخطوة للطلاب والمقيمين.
AGENTS.md	سجل توثيق أوامر ومهام وكلاء الذكاء الاصطناعي (AI Chronological Log).	- سجل رسمي يُوْرخ كافة المراحل المنفذة منذ بداية إنشاء المشروع. - توثيق أوامر الترحيل إلى لارافل، إصلاحات الـ API، خوارزميات الاستوديو، وتقارير الـ PDF. - يضمن عدم ضياع السياق البرمجي والتاريخي بين مختلف المطورين والوكلاء.
AI_LOG.md	سجل الشفافية والتفكير النقدي لاستخدام الذكاء الاصطناعي الأكاديمي.	- يوثق كل جلسة استخدام ذكاء اصطناعي لكل طالب من أعضاء الفريق. - يحتوي على مدخل الطالب أحمد القاضي (تأسيس خادم Flask ورفض Django). - يثبت للأستاذ أن الفريق يتخذ القرارات الهندسية بناءً على دراسة وتمحيص.
CONTRIBUTING.md	دليل إرشادات المساهمة للمطورين (Contribution Guidelines).	- قواعد إنشاء الفروع (Branching Naming Conventions). - معايير كتابة رسائل الـ Commits (Conventional Commits). - خطوات فتح طلبات الدمج (Pull Requests) ونماذج مراجعة الكود.
SECURITY.md	سياسة الأمان وحماية النظام (& Security Policy Vulnerability Reporting).	- جدول الإصدارات المدعومة بالتحديثات الأمنية. - آلية الإبلاغ الآمن عن الثغرات البرمجية دون النشر العلني. - تدابير حماية الـ CSRF والتحقق من قيود أحجام الصور المرفوعة.
CODE_OF_CONDUCT.md	ميثاق وقواعد السلوك المهني (Contributor Covenant Code of Conduct).	- معايير التعامل الأخلاقي والمهني بين أعضاء الفريق والمساهمين. - قواعد نبذ التعصب وتشجيع بيئة العمل الإيجابية والتعاون الأكاديمي.
CLAUDE.md	إرشادات وموجهات الوكلاء البرمجيين ومساعدتي التطوير.	- توجيهات مختصرة لبيئة التطوير وأوامر التشغيل ومسارات الخوادم لتسريع استجابة المساعدين الذكيين.

7. الدليل الكامل للعمل عبر Git و GitHub وتوثيق طلب الدمج #16 PR

يمثل Git نظام التحكم بالإصدارات المحلي، بينما يمثل GitHub المنصة السحابية لإدارة المشاريع والعمل الجماعي:

استراتيجية إدارة الفروع (Branching Strategy):

كل عضو في الفريق يمتلك فرعاً مستقلاً يبدأ بـ `/feature`، ويمنع الرفع المباشر إلى الفرع الرئيسي `main` إلا من خلال طلب دمج (Pull Request) يوافق عليه المشرف:

- فرع الواجهات المخصص لك: `feature/frontend-studio`
- الفرع الرئيسي للمشروع: `main`

أهم أوامر Git العملية خطوة بخطوة:

```
# 1. فحص حالة الملفات والتأكد من نظافة مساحة العمل
git status

# 2. إضافة الملفات المعدلة
git add resources/ public/ python_engine/ start_studio.bat

# 3. توثيق التعديلات برسالة واضحة
git commit -m "fix(frontend): route API via Laravel gateway, fix DOM references, add error handling and Al

# 4. رفع التعديلات لفرعك على المستودع السحابي
git push -u origin feature/frontend-studio

# 5. سحب آخر تحديثات المشرف وتفايدي التعارضات
git fetch origin
git merge origin/main
```

توثيق طلب الدمج الحالي المفتوح: Pull Request #16

تفاصيل طلب الدمج المسجل رسمياً باسم أحمد القاضي:

- رقم الطلب: Pull Request #16

- العنوان: تطوير وإصلاح الواجهة الأمامية: ربط الـ API ببوابة لارافل، تصحيح أخطاء الـ DOM، وبناء استوديو الواقع المعزز

- الفرع المصدري: `feature/frontend-studio` → الفرع المستهدف: `main`

- حالة الدمج: **Able to merge** (متوافق بنسبة 100% وبدون أي تعارضات - 0 Conflicts)

- الكوميترات الثلاثة المضمنة في الطلب:

1. `a579cae`: إصلاحات الفرونت إند وتوجيه الـ API وتصحيح مراجع الـ DOM ومودال الواقع المعزز.

2. `5729f3b`: تطوير التشغيل التلقائي عبر `run.py` ونقل المستخدم فوراً لصفحة التصميم.

3. `69d3ab1`: مزامنة الفرع مع آخر تحديثات المستودع وحل كافة التعارضات.

8. الشرح المعماري لبقية ملفات المشروع (لاراغل، بايثون، وقاعدة البيانات)

تكون على دراية كاملة بالمعمارية المتكاملة للنظام والرد على أي سؤال يخص تدفق البيانات:

المكون / المسار	التقنية	الوظيفة في معمارية النظام
<code>routes/web.php</code>	Laravel Routing	يستقبل طلبات المتصفح على <code>/</code> ويعرض قالب <code>welcome.blade.php</code> .
<code>routes/api.php</code>	Laravel API Routing	يعرّف مسار <code>POST /api/process</code> ومسار فحص الصحة <code>GET /api/health</code> .
<code>ImageProcessingController.php</code>	Laravel Controller	يعمل كوسيط عكسي (Reverse Proxy) ينقل الطلبات داخلياً إلى بايثون مع مهلة 60 ثانية للعمليات الكبيرة.
<code>database/database.sqlite</code>	SQLite Engine	قاعدة البيانات المحلية لتخزين جلسات العمل (Sessions) وسجلات المستخدمين وتوكنات الحماية.
<code>/python_engine/core</code>	NumPy / OpenCV	حزمة الخوارزميات الرياضية العلمية: العمليات النقطية، الفلاتر المكانية، تحويل فورييه 2D-FFT، والمورفولوجيا.
<code>composer.json</code>	PHP Package Manager	ملف إدارة مكتبات بيئة لاراغل وحزم الـ HTTP Client والـ Framework.
<code>package.json</code>	Node / Vite	ملف إدارة حزم الواجهة الأمامية وإعدادات تجميع الأصول (Assets Bundler).
<code>env.</code>	Environment Config	ملف متغيرات البيئة المحلية (منافذ الخوادم، مفتاح تشفير التطبيق، APP_KEY، واتصال قاعدة البيانات).

9. بنك الأسئلة المتوقعة في مناقشة المشروع وإجاباتها النموذجية

س 1: ما هو دورك المحدد في هذا المشروع البرمجي المشترك؟

الإجابة النموذجية: "دوري هو مهندس الواجهات وتجربة المستخدم (Frontend Developer). قمت ببناء استوديو المعالجة الرقمية بمحاكاة فوتوشوب، وتطوير محرك الـ Canvas والمقارنة الحية، واستوديو الواقع المعزز (AR Try-On)، وتأسيس مسارات الـ API في خادم بايثون، وربط الواجهة عبر طلبات AJAX آمنة مع نظام إشعارات حي، وإدارة فرع الواجهة في Git."

س 2: لماذا اخترتم خادم Flask في بايثون بدلاً من دمج الكود داخل PHP أو استخدام Django؟

الإجابة النموذجية: "وفقاً للتحليل الموثق في الـ AI Log، تم رفض دمج بايثون داخل PHP لأن حزمة php-python غير مستقرة وبطيئة في تمرير المصفوفات لـ OpenCV. وتم رفض Django لأنه إطار ضخم يفرض ميزات إضافية ثقيلة (Admin و ORM) لا نحتاجها، بينما Flask خفيف جداً وأسرع لمعالجة مصفوفات البكسلات الرقمية كخدمة مصغرة (Microservice)."

س 3: كيف تعمل ميزة مقارنة الصور (Split Canvas Slider) برمجياً؟

الإجابة النموذجية: "نستخدم عنصري Canvas متطابقين تماماً في الأبعاد، ونقوم بالتحكم في شريحة العرض عبر تعديل عرض الحاوية العلوية أو الـ Clip-Path بناءً على نسبة موضع الفأرة من 0% إلى 100% لحظياً دون أي إعادة تحميل للصفحة."

س 4: ما هي الآلية المتبعة لدمج التعديلات بين أعضاء الفريق في Git؟

الإجابة النموذجية: "اعتمدنا نموذج الفروع الوظيفية (Feature Branch Workflow). كل طالب يعمل على فرعه الخاص (`feature/frontend-studio`)، وعند اكتمال الميزة يتم فتح طلب دمج (Pull Request) ومراجعته من المشرف ودمجه في فرع `main` بدون أي تعارضات (Conflicts 0) كما هو موثق في طلب الدمج رقم #16."

س 5: كيف قمت بحل مشكلة الـ CORS وحماية الطلبات؟

الإجابة النموذجية: "قمنا بتوحيد كافة الطلبات عبر بوابة لاراغل الداخلية (`api/process/`) التي تعمل كوسيط عكسي (Reverse Proxy)، وقمنا بتمرير رمز الأمان `X-CSRF-TOKEN` في رأس الطلب لمنع هجمات التزوير وحل قيود الـ CORS نهائياً."

تم بحمد الله وتوفيره إعداد هذا المرجع الأكاديمي الشامل ليكون عوناً وسنداً للطلاب في مسيرته الأكاديمية والمهنية.